

Topic: Multi-Task Deep Learning for Predicting Chess Puzzle Difficulty and Tactical Pattern from Board Positions

Why choose this topic?

This topic is interesting to me because I am personally learning how to play chess, so I already have a curiosity about how chess positions and tactical patterns work. I have often heard about AI and deep learning models that can play chess at a very high level and even defeat human players, but I had never studied in detail how such models understand chess positions. That made me want to explore the topic through this project.

I chose chess pattern prediction because it connects personal interest with a meaningful deep learning problem. Instead of only looking at chess as a game, this project lets me study how a model can learn tactical structures such as forks, pins, skewers, and mates directly from board positions. This makes the project both personally motivating and technically challenging, since it combines my interest in improving at chess with my interest in understanding how deep learning can recognize complex patterns.

Why does this topic require deep learning to solve the problem?

Deep learning is appropriate because the input is a structured board position with spatial relationships between pieces. Convolutional neural networks are especially suitable for this kind of data because they can learn local and global spatial patterns from board tensors.

A simpler approach, such as linear regression or manually engineered rules, would likely miss many of the complex interactions between tactical motifs and difficulty. Traditional machine learning methods can still be useful, especially with hand-crafted chess features, but they usually depend heavily on feature design. Deep learning has the advantage of learning useful representations automatically from the board itself.

Strengths of the deep learning approach:

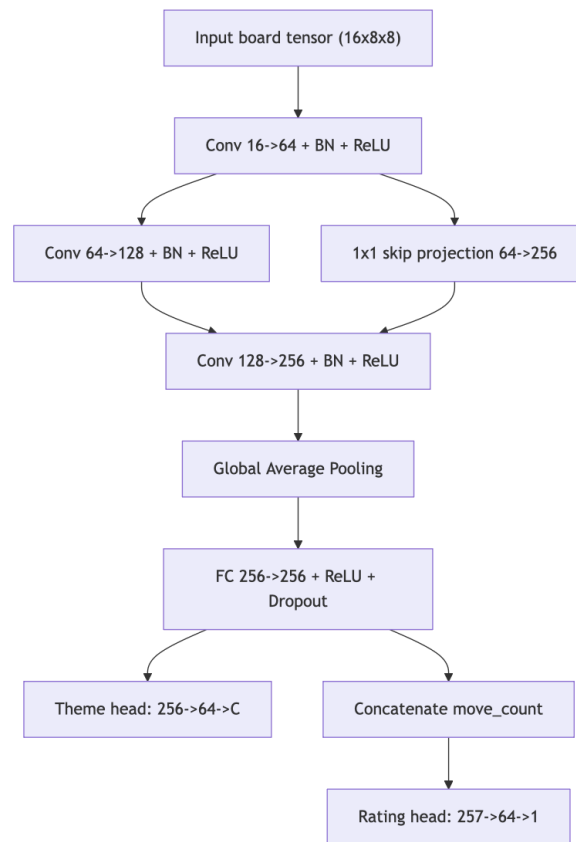
- Learn spatial tactical structure directly from board states.
- Can share features across related tasks.
- Reduces dependence on manual feature engineering.
- Supports end-to-end training.

Weaknesses of the deep learning approach:

- Requires more computation and more data than simpler models.
- Harder to interpret than rule-based systems.
- Performance depends on careful architecture and hyperparameter choices.
- Still may not capture long tactical sequences as well as full search-based chess engines.

Deep Learning Architecture

The project uses CNN-based models with board tensors of shape (16, 8, 8).



Input Representation

Each chess position in FEN format is converted into 16 planes:

- 12 piece planes for white and black pieces
- 1 plane for side to move
- 1 plane for white castling rights
- 1 plane for black castling rights
- 1 plane for normalized material balance

An additional scalar feature, move count, is also used by the CNN rating head.

1. SingleTaskCNN: This model predicts only puzzle ratings.
2. MultiTaskCNN: This model uses a shared CNN backbone and has two output heads:
 - Rating head for regression
 - Theme head for classification

Main Architecture

Shared backbone:

- Conv2d 16: 64, kernel 3x3, BatchNorm, ReLU
- Conv2d 64: 128, kernel 3x3, BatchNorm, ReLU
- Conv2d 128: 256, kernel 3x3, BatchNorm, ReLU
- Skip connection from first block to third block through 1x1 convolution
- Global average pooling
- Fully connected layer 256: 256
- ReLU

- Dropout

Rating head:

- Concatenate backbone features with move count
- Linear 257: 64
- ReLU
- Linear 64: 1

Theme head:

- Linear 256: 64
- ReLU
- Dropout
- Linear 64: number_of_themes

Code Explanation

- Data Handling: **dataset.py**
 - Loads the Lichess puzzle dataset
 - Filters the puzzles by rating and popularity
 - Selects themes from a controlled subset
 - Converts FEN positions into 16-plane tensors
 - Encodes labels
 - Builds PyTorch Dataset and DataLoader objects

Important preprocessing used in the notebook:

- Rating range: 1000-2500
- Minimum plays: 50
- Total sampled dataset size: 50,000
- Balanced sampling across themes

- Model Definition: **models.py**

This file defines: ChessCNNBackbone, SingleTaskCNN and MultiTaskCNN

The multi-task model is the main contribution of the project because it is the custom architecture designed to answer the research question.

Training: **train.py**

This file:

- Runs training and validation loops
- Uses MSELoss for rating regression
- Uses CrossEntropyLoss for theme classification
- Combines them into a multi-task loss:

$$L = \text{lambda_rating} * L_MSE + \text{lambda_theme} * L_CE$$
- Uses Adam optimizer
- Uses gradient clipping
- Uses learning rate scheduling
- Uses early stopping
- Saves the best model checkpoint

- Evaluation: **evaluate.py**

This file computes: Rating MAE, Rating RMSE, Theme accuracy, Theme F1 score and Classification report for tactical motif prediction

- Experiment Scripts: **experiment.py, ablation.py, and run_experiment.py**

These scripts organize experiments, run ablation studies, and compare models.

Training Method, Dataset, and Source

The dataset comes from the official Lichess puzzle database through Hugging Face. The full dataset contains 5,829,565 puzzles.

For the main experiment, the notebook filtered:

- Only ratings between 1000 and 2500
- Only puzzles with at least 50 plays
- Only puzzles whose main theme belongs to: fork, pin, skewer and mate

After filtering and balanced sampling, the final dataset contained 50,000 puzzles:

- 12,500 fork
- 12,500 pin
- 12,500 skewer
- 12,500 mate

The model was trained on GPU (cuda) for up to 30 epochs with early stopping. Validation MAE was used to select the best checkpoint.

Model Evaluation

The project evaluates both regression and classification performance.

Metrics

For rating prediction:

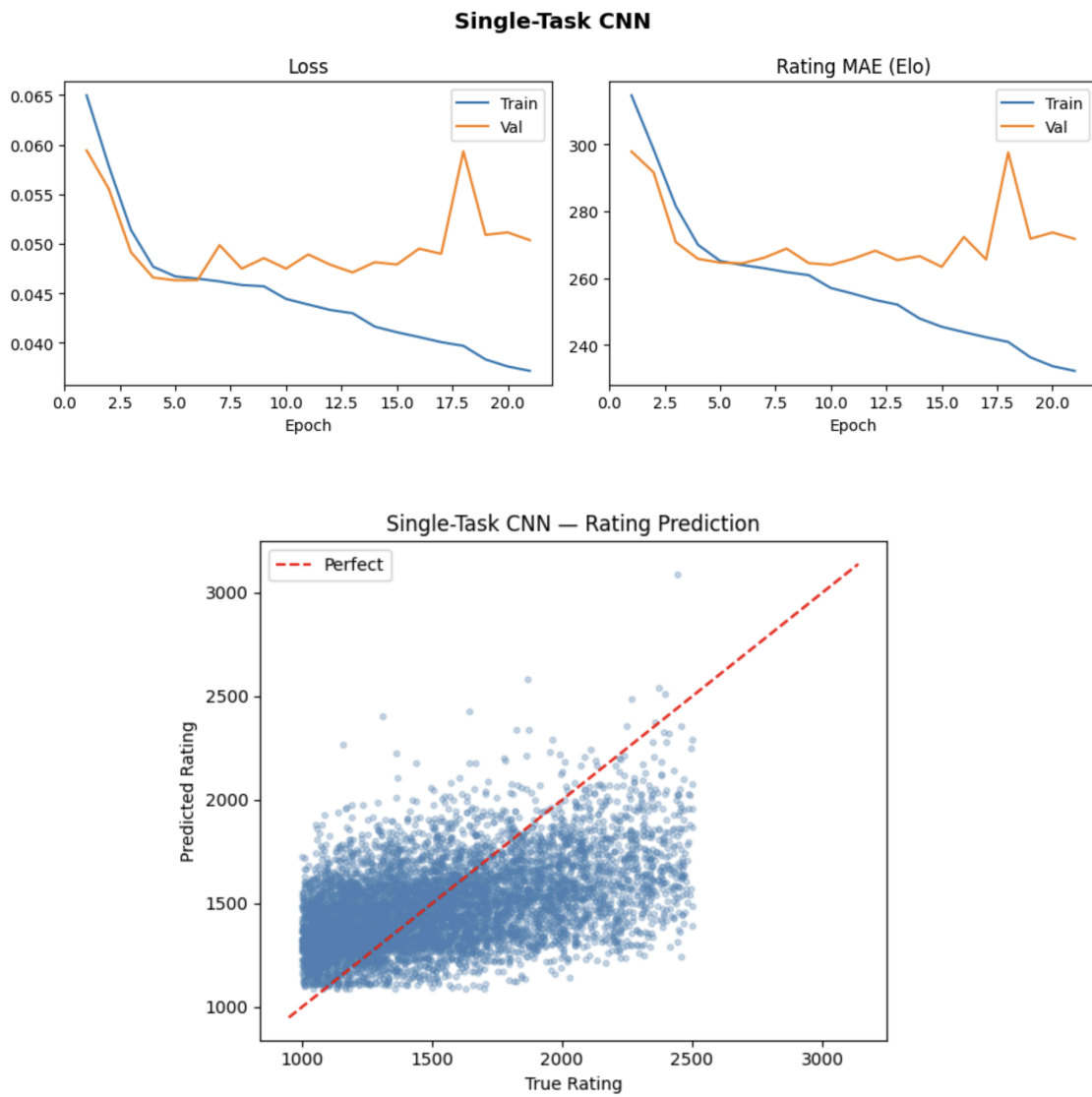
- MAE in Elo points
- RMSE in Elo points

For theme classification:

- Accuracy
- Precision
- Recall
- F1-score

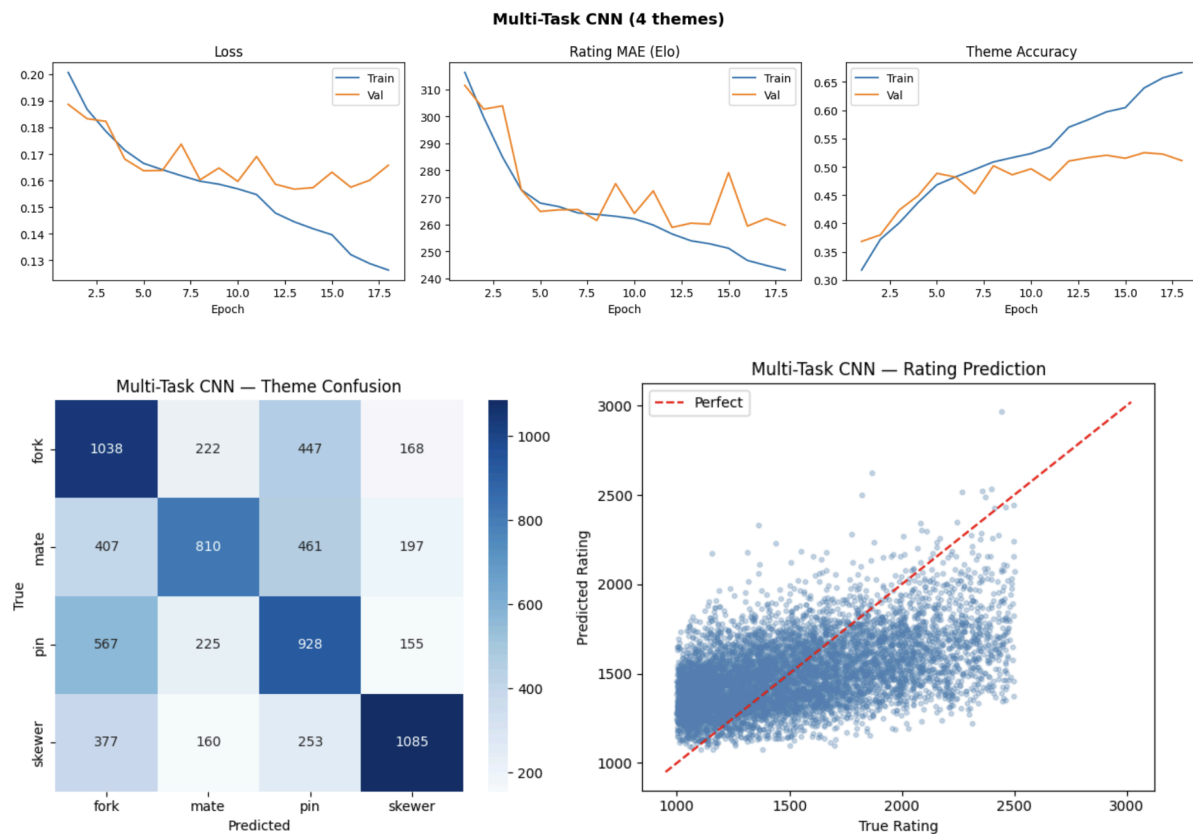
Main Results

Single-Task CNN



- Test MAE: 263.60 Elo
- Test RMSE: 331.74 Elo

Multi-Task CNN (4 themes)



- Test MAE: 261.09 Elo
- Test RMSE: 324.85 Elo
- Theme accuracy: 0.5148
- Theme macro F1: 0.5177

Interpretation

The multi-task model slightly outperformed the single-task CNN on rating prediction:

- MAE improved from 263.60 to 261.09
- RMSE improved from 331.74 to 324.85

This suggests that learning tactical motif classification provides useful shared information for estimating puzzle difficulty. The gain is modest, but it supports the project's main hypothesis.

Classification Report for 4 Themes

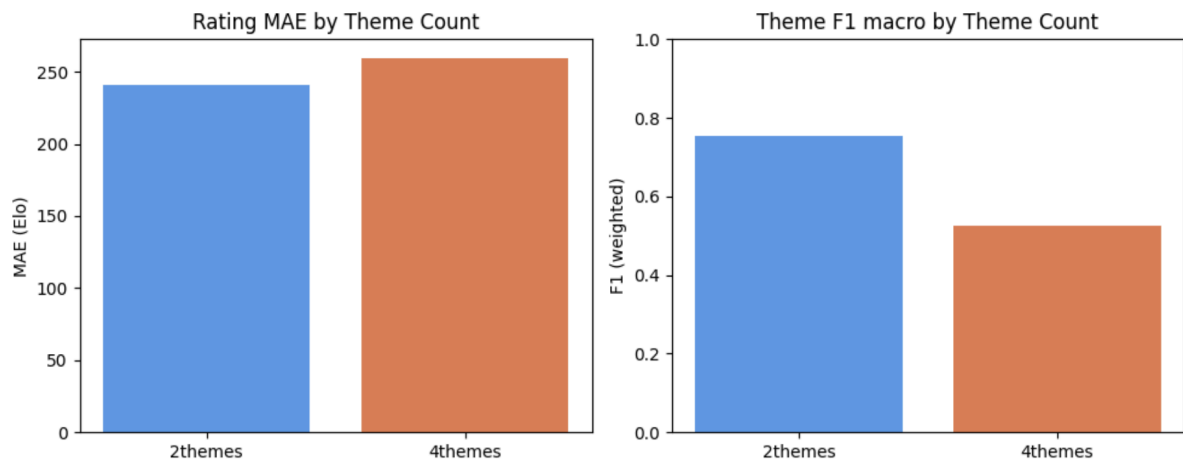
Per-class results:

- fork: F1 = 0.49
- mate: F1 = 0.49
- pin: F1 = 0.47
- skewer: F1 = 0.62

The model performed best on skewer, while fork, mate, and pin were more difficult to separate. This is reasonable because some tactical motifs overlap and may share visual patterns.

Ablation Study

An ablation study compared multi-task learning with 2 themes versus 4 themes.



2-Theme Setting

Themes: fork and mate

Results:

- Rating MAE: 240.93 Elo
- Rating RMSE: 304.41 Elo
- Theme accuracy: 0.7549
- Theme macro F1: 0.7549

4-Theme Setting

Themes: fork, pin, skewer and mate

Results:

- Rating MAE: 261.09 Elo
- Rating RMSE: 324.85 Elo
- Theme accuracy: 0.5148
- Theme macro F1: 0.5177

Ablation Interpretation

The 2-theme setting performed much better than the 4-theme setting because:

- The classification problem is easier with fewer classes.
- Themes such as pin and skewer can be visually similar.
- Increasing label diversity makes both classification and shared representation learning harder.

Even so, the 4-theme multi-task model still slightly improved rating prediction compared with the single-task model, which supports the idea that tactical supervision is useful.

Strengths and Weaknesses of the Final System

Strengths

- Uses a custom deep learning design rather than only off-the-shelf models.
- Tackles two related tasks jointly.
- Uses a large real-world chess dataset.
- Includes a clean comparison between single-task and multi-task learning.

- Includes an ablation study with different theme settings.

Weaknesses

- Improvement over the single-task model is relatively small.
- The model only sees static board positions and not full search trees.
- Theme classification accuracy for 4 classes is moderate.
- The proposed MLP baseline appears in the design but is not included in the final result table from the notebook.

Conclusion

This project developed a multi-task deep learning system for chess puzzle analysis using PyTorch and the Lichess puzzle dataset. The model predicts both puzzle difficulty and tactical motif from board positions encoded as multi-plane tensors.

The results show that the multi-task CNN slightly improves rating prediction over a single-task CNN, reducing test MAE from 263.60 Elo to 261.09 Elo and test RMSE from 331.74 Elo to 324.85 Elo. The model also achieved 51.48% theme classification accuracy and 0.5177 macro F1 across four tactical classes.

The ablation study showed that reducing the number of classes from 4 themes to 2 themes made the task much easier, improving both regression and classification results. Overall, the project supports the idea that shared tactical learning can help puzzle difficulty prediction, although the benefit is moderate and there is still room for improvement.

Reference Articles and Related Work

- Lichess puzzle dataset: <https://huggingface.co/datasets/Lichess/chess-puzzles>
- Introduction to Multi-Task Learning(MTL) for Deep Learning: <https://www.geeksforgeeks.org/deep-learning/introduction-to-multi-task-learningmtl-for-deep-learning/>
- DeepChess: End-to-End Deep Neural Network for Automatic Learning in Chess: <https://www.cs.tau.ac.il/~wolf/papers/deepchess.pdf>

Group Contribution: This project was completed individually.

GitHub: <https://github.com/1stChaS/Deep-Learning-for-Predicting-Chess-Puzzle.git>